

ENSTEIN ROBOTS & AUTOMATIONS PVT. LTD.

Production Line Monitor

Complete Project Documentation

Software Design · Architecture · Source Reference · Deployment & Operations

Document Type	Software Design & Deployment Manual
System	Production Line Monitor (Camera Inspection / GPIO OK-NG Control)
Platform	Raspberry Pi 4 — Python / PySide6 (Qt Quick)
Version	3.0
Date	21 August 2026

Table of Contents

Part I — Software Design Documentation	4
1. Project Overview	5
2. System Objective	7
3. Software Architecture	9
4. Software Dependencies	11
5. Module-to-Module Dependency Map	13
6. Files, Classes & Function Reference	15
7. Application Execution Flow	29
8. UI-to-Code Mapping & Data Flow	31
9. UI Walkthrough — QML Screens	35
10. Hardware Reference	37
11. Configuration Reference	40
Part II — Deployment & Operations Guide	41
12. Raspberry Pi OS Installation & System Prerequisites	42
13. Setup and Run (Development)	44
14. App Creation (Build Standalone Executable)	47
15. Autostart the Application (Kiosk Mode)	49
16. Boot Screen Customization	51
17. Hide the Taskbar / Panel	53
18. Passwordless Shutdown / Reboot Buttons	56
19. Error Log & Troubleshooting	57

20. Updating an Existing Install 59

21. Complete Project Working Flow — Build to Cleanup 60

Part I — Software Design Documentation

Objective · Architecture · Dependencies · Source Reference · Data Flow

1. Project Overview

The Production Line Monitor is a desktop application that inspects parts on a production line using a camera, receives pass/fail results over a serial connection, logs the results, and signals the line controller (OK/NG) via GPIO on a Raspberry Pi.

Technology Stack

- Python
- PySide6 / Qt Quick (QML) for the user interface
- `gpio` for GPIO control
- `pyserial` for the camera serial link
- PyInstaller for building a standalone executable

Repository Layout

Path	Purpose
<code>main.py</code>	Application entry point
<code>backend.py</code>	QObject bridge exposing app state and actions to QML
<code>gpio_controller.py</code>	GPIO pin setup and OK/NG/conveyor control
<code>serial_receiver.py</code>	Background thread handling the camera serial connection
<code>Production_model.py</code>	Per-part production counters
<code>LogModel.py</code>	Application log model / file logging
<code>paths.py</code>	Runtime path resolution
<code>tech/</code>	QML UI files (<code>Main.qml</code> , <code>Part1-4.qml</code> , etc.)
<code>config.json</code>	Serial port / baud rate configuration
<code>counts.json</code>	Persisted production counts

<code>main.spec</code>	PyInstaller build spec
<code>deploy/</code>	Raspberry Pi deployment assets and guide

Development Environment

The source code itself is written and maintained on an **Ubuntu 26.04 LTS ("Resolute Raccoon") x86_64 workstation** — not on the Raspberry Pi. The Pi (Part II of this document) is the deployment and runtime target only. This split is possible because GPIO and serial I/O are both optional at startup (Section 2): on this Ubuntu workstation, `gpiod` has no `/dev/gpiochip0` to attach to and no camera is attached, so `GPIOController` and `SerialReceiver` fall back to their disabled/inert modes automatically (Section 6) — the UI and application logic can be written, run, and tested here directly with `python main.py` (Section 13), with `SerialReceiver.simulateResult()` available to inject fake camera results for that testing. Real hardware behavior (GPIO timing, the actual camera link) can only be verified on the Pi itself.

2. System Objective

This section states the problem the system exists to solve and the concrete objectives that drove its design, independent of implementation detail.

Problem

Parts moving through this production line previously relied on manual, visual pass/fail judgement. That is inconsistent between operators, leaves no auditable record, and gives the line controller no reliable electrical signal to act on. The Production Line Monitor replaces that manual step with a camera-based inspection, and converts the camera's result into both a physical signal the existing line hardware already understands (OK/NG on GPIO) and a persistent digital record.

Objectives

- **Automate the pass/fail decision** using an external machine-vision camera unit rather than operator judgement, per part, per inspection scheme (up to 4 part types).
- **Signal the result to the physical line controller** in the timing it expects — OK/NG held on GPIO until the part physically clears the sensor, not a fixed-width pulse (see Section 10).
- **Keep a persistent, crash-safe audit trail** — OK/NG/Total counts per part and the most recent results, saved to disk after every single result so a power loss never loses more than the in-flight inspection.
- **Minimize operator burden** — a touch-first kiosk interface with one screen per part type, three buttons (Start/Stop/Reset), and a Report screen; no keyboard or desktop interaction expected.
- **Run unattended on a Raspberry Pi 4** — boots directly into the application, fullscreen, with no visible desktop or login prompt (Part II covers this in full).
- **Degrade gracefully off the physical hardware** — both GPIO and serial I/O are optional at startup, so the same codebase runs (with those features inert) on a development machine with no camera or GPIO attached.

Scope

Single-station part inspection: one camera, one serial link, one GPIO line-controller interface, up

to four inspection schemes ("Part 1"– "Part 4"). Out of scope: multi-station coordination, remote/networked monitoring, and any inspection logic itself — the pass/fail decision is made entirely by the external camera unit; this system only acts on the result it reports.

3. Software Architecture

The application is organized in layers, each owning exactly one concern. Every layer below the QML UI is a plain `QObject`, which is what lets Qt's signal/slot mechanism carry events across thread and layer boundaries safely.



4. Software Dependencies

Python Package Dependencies (`requirements.txt`)

Package	Version	Purpose	Notes
PySide6	6.10.2	Qt Quick/QML UI framework and the application event loop	Bundled into the PyInstaller binary; the heaviest dependency, and the main driver of the app's several-second cold-start time on the Pi 4
pyserial	3.5	Serial port communication with the camera	Runs entirely on the background <code>SerialWorker</code> thread (Section 6) — never blocks the GUI thread
gpio	>=2.1, <3	Linux GPIO character-device access (v2 API)	Requires the running user to be in the <code>gpio</code> group (Section 13/14); optional at runtime — <code>GPIOController</code> disables itself gracefully if it is missing or fails to initialize. See Section 12 for the system-level <code>libgpio-dev</code> package this may need to compile.

Development-Only Dependency

Tool	Purpose	Notes
PyInstaller	Packages <code>main.py</code> and its dependencies into the standalone <code>dist/main</code> binary	Not listed in <code>requirements.txt</code> ; installed separately in Section 14. Not a runtime dependency of the running application itself.

System-Level Dependencies

- **Python 3** — this project's local virtual environments target 3.12/3.13; no strict minimum is pinned in the repository.
- `/dev/gpiochip0` — the Linux GPIO character device, standard on Raspberry Pi OS; `gpio` talks to it directly.

- **USB-serial driver** — the camera's adapter must enumerate as a `/dev/ttyUSB*` device (kernel `usbserial`/vendor driver, already present on Raspberry Pi OS).
- **Qt platform plugin** — PyInstaller bundles the X11 (`xcb`) Qt platform plugin required to show a window under the Pi's X11/LXDE desktop (see Part II).

Section 12 covers installing every one of these system-level prerequisites on a completely fresh Raspberry Pi OS image, before any of the above Python-level installs are attempted.

5. Module-to-Module Dependency Map

The internal import graph is deliberately shallow. Only one file imports another project module for its own logic (`Production_model.py` → `paths.py`); every other cross-module relationship is wiring done once, from outside, in `main.py`.

Module	Imports (internal)	Depended On By	Notes
<code>main.py</code>	<code>backend</code> , <code>Production_model</code> , <code>serial_receiver</code> , <code>gpio_controller</code> , <code>LogModel</code> , <code>paths</code>	— (entry point)	The composition root: the only file that constructs all five collaborators and wires them together
<code>backend.py</code>	none internal	<code>main.py</code>	Receives all four collaborators via constructor injection; calls only their public methods/signals, never their internals — any one could be swapped for a test double
<code>gpio_controller.py</code>	none internal	<code>main.py</code> , <code>backend.py</code> , QML (context property <code>gpiocontroller</code>)	Self-contained; QML calls <code>conveyor_on()</code> / <code>conveyor_off()</code> on it directly, bypassing Backend entirely
<code>serial_receiver.py</code>	none internal	<code>main.py</code> , <code>backend.py</code>	Self-contained; not exposed to QML directly, only reachable through Backend's <code>triggerCamera()</code> /s

			<code>electScheme()</code>
<code>Production_model.py</code>	<code>paths.py</code>	<code>main.py, backend.py</code>	The one internal cross-module dependency in the codebase — needs <code>app_dir()</code> to locate <code>counts.json</code>
<code>LogModel.py</code>	none internal	<code>main.py, backend.py, QML (context property LogModel)</code>	Exposed directly to QML as well as to Backend
<code>paths.py</code>	none (leaf module)	<code>Production_model.py, main.py</code>	Every file that touches persistence ultimately depends on this one function, <code>app_dir()</code>

Notice what is *not* in this table: `gpio_controller.py` and `serial_receiver.py` never import each other, and neither imports `Production_model.py` or `LogModel.py`. All cross-concern coordination (sensor → camera trigger, camera result → GPIO + persisted log) happens exclusively through signals wired up in `Backend.__init__`. This is why Section 8's data-flow walkthrough routes everything through Backend.

6. Files, Classes & Function Reference

This section is the formal reference for every Python source file: its role, the classes it defines, and every method/function on those classes. Parameters and return types reflect the actual source, not a simplified summary.

`paths.py`

Role: the single source of truth for "where does the app live," used by everything that must persist data next to the running app.

Function	Parameters	Returns	Description
<code>app_dir()</code>	none	Path	The PyInstaller executable's own folder when frozen (<code>sys.frozen</code>), otherwise this source file's folder. PyInstaller's bundle-extraction directory is temporary/read-only, so anything that must survive a restart (<code>config.json</code> , <code>counts.json</code> , <code>logs/</code>) resolves its path through this function.

`main.py`

Role: composition root and application entry point — no reusable logic lives here, only startup wiring.

Class `_StreamToLog`

Method	Parameters	Returns	Description
<code>__init__</code>	<code>logger</code> , <code>level</code> , <code>original_stream</code>	—	Stores the target logger/level and the original stream to mirror to
<code>write</code>	<code>message</code>	None	Writes to the original stream (so a terminal run still shows live output) <i>and</i> logs the stripped message, so nothing is lost when the app runs unattended with no

visible console

<code>flush</code>	<code>none</code>	<code>None</code>	Flushes the original stream
--------------------	-------------------	-------------------	-----------------------------

Module-Level Functions

Function	Parameters	Returns	Description
<code>setup_error_logging()</code>	<code>none</code>	<code>None</code>	Creates a <code>RotatingFileHandler</code> at <code>logs/app.log</code> (2MB, 3 backups); redirects <code>sys.stdout/sys.stderr</code> through <code>_StreamToLog</code> ; installs a global <code>sys.excepthook</code> so an uncaught exception is logged, not just silently lost
<code>load_config()</code>	<code>none</code>	<code>dict</code>	Reads <code>config.json</code> next to the app, merged over the defaults <code>{"serial_port": "/dev/ttyUSB0", "baudrate": 9600}</code> ; tolerates a missing or invalid file

`gpio_controller.py`

Role: owns every physical GPIO line — the sensor input and the OK/NG/conveyor outputs.

Class `GPIOController(QObject)`

Signals: `sensorDetected()`, `cameraTriggered()` (alias `conveyorShorten`), `okSignal()`, `ngSignal()`. Constants: `SENSOR_PIN=17`, `CONVEYOR_SHORTEN_PIN=24`, `OK_PIN=22`, `NG_PIN=23`, `CONVEYOR_PIN=27`, `PULSE_TIME_MS=100`.

Method	Parameters	Returns	Description
<code>__init__</code>	<code>enable_gpio=True</code>	—	Requests all five GPIO lines via <code>gpio.request_lines</code> ; on any failure (missing <code>gpio</code> , no hardware) sets <code>enable_gpio=False</code> and every method below becomes a safe no-op. On success, starts a 50ms <code>QTimer</code> polling

			<code>checkSensor()</code> .
<code>read_sensor()</code>	none	bool	Reads <code>SENSOR_PIN</code> 's current value
			Core state machine, called every 50ms. On the inactive → active transition: emits <code>sensorDetected</code> , turns conveyor-shorten on. On the part clearing: turns conveyor-shorten off, and if a result is latched (<code>result_active</code>), clears <code>OK_PIN/NG_PIN</code> back off — this is the exact mechanism behind the OK/NG "hold until sensor clears" behavior (Section 10). Fully annotated below.
<code>checkSensor()</code>	none	None	
<code>conveyor_shorten_on()</code>	none	None	Drives <code>CONVEYOR_SHORTEN_PIN</code> and emits <code>conveyorShorten</code>
<code>conveyor_shorten_off()</code>	none	None	Restores <code>CONVEYOR_SHORTEN_PIN</code> and emits <code>conveyorShorten</code>
<code>set_ok()</code>	none	None	Forces <code>NG_PIN</code> off, drives <code>OK_PIN</code> on, sets <code>result_active=True</code> , emits <code>okSignal</code>
<code>ok_off()</code>	none	None	Restores <code>OK_PIN</code> off. Present but not currently called — turning OK back off happens inside <code>checkSensor()</code> , not here.
<code>set_ng()</code>	none	None	Forces <code>OK_PIN</code> off, drives <code>NG_PIN</code> on, sets <code>result_active=True</code> , emits <code>ngSignal</code>
<code>ng_off()</code>	none	None	Restores <code>NG_PIN</code> off. Same as <code>ok_off()</code> : present but currently unused.
<code>conveyor_on()</code> <i>[Slot]</i>	none	None	Drives <code>CONVEYOR_PIN</code> on — called directly from QML's START button
<code>conveyor_off()</code> <i>[Slot]</i>	none	None	Restores <code>CONVEYOR_PIN</code> off — called directly from QML's STOP/BACK buttons
<code>all_off()</code>	none	None	Drives every output pin back to its off state

<code>cleanup()</code>	<code>none</code>	<code>None</code>	Stops the sensor timer, calls <code>all_off()</code> , releases the <code>gpio</code> line request. Wired to <code>app.aboutToQuit</code> in <code>main.py</code> .
------------------------	-------------------	-------------------	---

`serial_receiver.py`

- Role: owns the camera serial link. Splits blocking I/O (worker thread) from protocol/state handling (GUI thread) so a wedged USB-serial adapter can never freeze the UI.

Class `SerialWorker(QObject)` — runs on a background `QThread`

Signals: `connected()`, `disconnected(str)`, `lineReceived(str)`, `heartbeat()`, `workerFinished()`.

Method	Parameters	Returns	Description
<code>__init__</code>	<code>port</code> , <code>baudrate</code>	—	Stores connection settings; creates the thread-safe write queue
<code>run()</code>	<code>none</code>	<code>None</code>	The main loop (connected to <code>thread.started</code>): opens the port, then repeatedly flushes queued writes, blocking-reads one line (0.2s timeout), and emits <code>heartbeat</code> every pass plus <code>lineReceived</code> if a non-empty line arrived
<code>_sleep(seconds)</code>	<code>seconds</code>	<code>None</code>	Interruptible sleep used between reconnect attempts
<code>_flushPendingWrites(ser)</code>	<code>ser</code>	<code>bool</code>	Sends any queued commands before the next blocking read; returns <code>False</code> (and closes the port) if a write fails
<code>_open()</code>	<code>none</code>	<code>None</code>	Opens the <code>serial.Serial</code> connection (8N1, 0.2s timeout); emits <code>connected</code> or <code>disconnected</code>
<code>_close(reason="")</code>	<code>reason</code>	<code>None</code>	Closes the port and emits

			<code>disconnected(reason)</code>
<code>stop()</code>	none	None	Called from the GUI thread: stops the loop and cancels any in-flight read
<code>kick()</code>	none	None	Called by the watchdog: sets a force-reconnect flag and cancels the current read/write to unstick a wedged port at the OS/driver level
<code>queueWrite(data: bytes)</code>	data	None	Thread-safe: queues a command for the worker thread to actually send

Class `SerialReceiver(QObject)` — lives on the GUI thread

Signals: `resultReceived(str, str)`, `errorOccurred(str)`, `switchReady(bool)`, `connectionChanged(bool)`. Constants: `SWITCH_COOLDOWN_SECONDS=0.5`. `SCHEMES = {1:"S PART1", 2:"S PART2", 3:"S PART3", 4:"S PART4"}`.

Method	Parameters	Returns	Description
<code>__init__</code>	<code>port="/dev/ttyUSB0", baudrate=9600</code>	—	Creates the worker thread and a 1s watchdog timer; does not start either
<code>connectSerial() [Slot]</code>	none	None	Starts the worker thread and the watchdog
<code>disconnectSerial() [Slot]</code>	none	None	Stops the watchdog, stops the worker, and waits (up to 2s) for the thread to quit cleanly
<code>_onConnected()</code>	none	None	Sets <code>isConnected=True</code> , emits <code>connectionChanged(True)</code>
<code>_onDisconnected(reason)</code>	reason	None	Emits <code>errorOccurred</code> if there was a reason, then <code>connectionChanged(False)</code>
<code>_onHeartbeat()</code>	none	None	Records the current time as the last

			heartbeat
<code>_checkWatchdog()</code>	<code>none</code>	<code>None</code>	If >30s have passed since the last heartbeat, calls <code>worker.kick()</code> to force a reconnect
<code>_onLine(data)</code>	<code>data</code>	<code>None</code>	The protocol parser — see Section 10 for the exact rules, and the annotated walkthrough below
<code>selectScheme(schemeNumber) [Slot]</code>	<code>schemeNumber: int</code>	<code>None</code>	Queues the matching <code>SCHEMES</code> command, starts the 0.5s cooldown, emits <code>switchReady(False)</code>
<code>trigger() [Slot]</code>	<code>none</code>	<code>None</code>	Queues the literal camera-capture command <code>b"T"</code>
<code>simulateResult(part, status) [Slot]</code>	<code>part: str,</code> <code>status: str</code>	<code>None</code>	Emits <code>resultReceived</code> directly with no real hardware — a hook for developing/testing the UI without a camera

backend.py

- Role: the single QML-facing hub. Every collaborator above is injected here and wired together;
- nothing outside this file coordinates more than one collaborator at once.

Class Backend(QObject)

Signals: `schemeReadyChanged(bool)`, `connectedChanged(bool)`, and `part{1-4}{Ok, Ng, Total}Changed(int)` (12 signals, one per part × counter).

Method	Parameters	Returns	Description
<code>__init__</code>	<code>productionModel</code> , <code>serialReceiver</code> , <code>gpioController</code> , <code>logModel</code>	—	Stores all four collaborators and connects: <code>productionModel.dataChanged</code> →emits signals; <code>serialReceiver.switchReady</code> → <code>_onSchemeReadyChanged</code> ;

			<code>serialReceiver.connectionChanged→</code> <code>_onConnectionChanged;</code> <code>serialReceiver.resultReceived→add</code> <code>Result</code> and <code>→handleGPIOResult;</code> <code>gpioController.sensorDetected→han</code> <code>dleSensor</code>
<code>getOk(part) [Slot]</code>	<code>part: int</code>	<code>int</code>	Delegates to <code>productionModel.getOK(part)</code>
<code>getNg(part) [Slot]</code>	<code>part: int</code>	<code>int</code>	Delegates to <code>productionModel.getNG(part)</code>
<code>getTotal(part) [Slot]</code>	<code>part: int</code>	<code>int</code>	Delegates to <code>productionModel.getTotal(part)</code>
<code>addResult(part, status) [Slot]</code>	<code>part: str,</code> <code>status: str</code>	<code>None</code>	Records the result into <code>productionModel</code> and into <code>logModel</code>
<code>handleSensor() [Slot]</code>	<code>none</code>	<code>None</code>	Calls <code>serialReceiver.trigger()</code> — the sensor-to-camera link
<code>handleGPIOResult(part, status) [Slot]</code>	<code>part: str,</code> <code>status: str</code>	<code>None</code>	Calls <code>gpioController.set_ok()</code> or <code>set_ng()</code> depending on <code>status</code>
<code>selectScheme(schemeNumber) [Slot]</code>	<code>schemeNumber: int</code>	<code>None</code>	Delegates to <code>serialReceiver.selectScheme()</code>
<code>schemeReady [Property, bool]</code>	—	<code>bool</code>	<code>self._schemeReady</code> and <code>self._connected</code> — combines both flags server-side so QML checks one property instead of two
<code>connected [Property, bool]</code>	—	<code>bool</code>	Current serial connection state
<code>triggerCamera() [Slot]</code>	<code>none</code>	<code>None</code>	Manual trigger, delegates to <code>serialReceiver.trigger()</code>
<code>shutdownSystem()</code>	<code>none</code>	<code>None</code>	Runs <code>sudo shutdown -h now</code> (needs

<code>[Slot]</code>			the passwordless rule from Section 18)
<code>rebootSystem()</code>	none	None	Runs <code>sudo reboot</code> (needs the passwordless rule from Section 18)
<code>[Slot]</code>			Strips PyInstaller's <code>__MEI*/*_PYI*</code> self-extraction env vars, then relaunches the same executable/interpreter with the same arguments and quits the current instance — no sudo needed. Not currently wired to the UI : relaunching this frozen build crashes (a PyInstaller onefile self-extraction race, still being tracked down), so the Restart menu item in <code>Main.qml</code> is commented out. The method and its env-var workaround are left in place for when that is resolved.
<code>restartApp()</code>	none	None	
<code>[Slot]</code>			Re-emits all 12 <code>part{Ok,Ng,Total}</code> Changed signals so every bound QML Label refreshes
<code>emitSignals()</code>	none	None	

Production_model.py

Role: owns every production count and the result log, and their persistence to `counts.json`.

Class `ProductionModel(QObject)`

Signal: `dataChanged()`.

Method	Parameters	Returns	Description
<code>__init__</code>	none	—	Initializes empty part/baseline dicts for parts 1–4, then calls <code>loadData()</code>
<code>loadData()</code>	none	None	Reads <code>counts.json</code> ; creates it via <code>saveData()</code> if missing; tolerant of a corrupt file

<code>saveData()</code>	<code>none</code>	<code>None</code>	Writes <code>parts</code> , <code>baseLines</code> , and <code>logs</code> to <code>counts.json</code>
<code>addLog(part, status) [Slot]</code>	<code>part: str,</code> <code>status: str</code>	<code>None</code>	Validates part/status, appends a timestamped log entry, increments the matching counter(s), saves immediately, emits <code>dataChanged</code>
<code>getOK(part) / getNG(part) /</code> <code>getTotal(part) [Slot]</code>	<code>part: int</code>	<code>int</code>	Raw lifetime counters for that part
<code>getOKSinceReset(part) /</code> <code>getNGSinceReset(part) /</code> <code>getTotalSinceReset(part)</code> <code>[Slot]</code>	<code>part: int</code>	<code>int</code>	Live counter minus that part's baseline — what the Part page tiles actually display
<code>resetPart(part) [Slot]</code>	<code>part: int</code>	<code>None</code>	Snapshots the current counts as the new baseline for that part (does <i>not</i> zero the raw counters)
<code>filterDate(fromDate,</code> <code>toDate) [Slot]</code>	<code>fromDate:</code> <code>str, toDate:</code> <code>str</code>	<code>None</code>	Recomputes the live <code>parts</code> counters from <code>logs</code> history for a date range — a display filter, mutates in place
<code>resetData() [Slot]</code>	<code>none</code>	<code>None</code>	The only full wipe: zeroes every part's counts and baselines, clears all logs
<code>getTotalOK() / getTotalNG() /</code> <code>getTotalParts()</code>	<code>none</code>	<code>int</code>	Sums across all four parts; exposed as the <code>totalOK/totalNG/totalParts</code> Properties

LogModel.py

- Role: the 5-most-recent-results list shown on every Part page.

Class `LogModel(QAbstractListModel)`

Roles: `TimeRole`, `EventRole`, `StatusRole` (exposed to QML as `time/event/status`). Signal: `latestChanged()`.

Method	Parameters	Returns	Description
<code>__init__</code>	<code>none</code>	—	Starts with an empty in-memory log list (not persisted independently — seeded from <code>counts.json</code> 's last 5 logs at startup, see Section 7)
<code>rowCount(parent)</code>	<code>parent</code>	<code>int</code>	Standard <code>QAbstractListModel</code> override
<code>data(index, role)</code>	<code>index, role</code>	<code>value</code> or <code>None</code>	Standard <code>QAbstractListModel</code> override
<code>roleNames()</code>	<code>none</code>	<code>dict</code>	Maps the three custom roles to their QML-visible names
<code>latestStatus / latestTime</code> <i>[Property, str]</i>	—	<code>str</code>	The newest entry's status/time, or <code>"-"</code> if empty — feeds the "LAST RESULT" tile
<code>addLog(time, event, status)</code>	<code>time: str,</code> <code>event: str,</code> <code>status: str</code>	<code>None</code>	Inserts at index 0 (newest first) via proper <code>beginInsertRows/endInsertRows</code> ; trims anything past index 5; emits <code>latestChanged</code>
<code>clear()</code> <i>[Slot]</i>	<code>none</code>	<code>None</code>	Empties the list via <code>beginRemoveRows/endRemoveRows</code> ; called by the Production Report screen's full Reset

Annotated Source Walkthroughs

The reference tables above answer "what does this function do." The two pieces of logic below are the least obvious in the whole codebase — this walks through the actual source line by line, so a reader who has never seen this project can verify the behavior directly rather than trust a paraphrase.

Walkthrough 1 — The OK/NG "hold until sensor clears" state machine (`gpio_controller.py`, `checkSensor()`)

```
def checkSensor(self):

    sensorState = self.read_sensor()

    # New product detected: LOW -> HIGH

    if not sensorState and not self.sensorTriggered:

        self.sensorTriggered = True

        # (A)

        self.sensorDetected.emit()

        self.conveyor_shorten_on()

    elif sensorState:

        if self.sensorTriggered:

            self.sensorTriggered = False

            self.conveyor_shorten_off()

        # (B)

        if self.result_active:

            self.request.set_value(self.OK_PIN, Value.ACTIVE)

            self.request.set_value(self.NG_PIN, Value.ACTIVE)

            self.result_active = False

    self.lastSensorState = sensorState
```

1. This method is called by a `QTimer` every 50ms — it is a poll loop, not an interrupt handler. `sensorState` is the raw current pin reading.

2. **Part arrives** (not `sensorState` and not `self.sensorTriggered`): this is the edge — the sensor just went active *and* it was not already flagged as triggered (so a part sitting under the sensor for multiple 50ms polls only fires this branch once). Line (A) is what starts the whole inspection cycle: `sensorDetected` is the signal `Backend.handleSensor()` listens for, which triggers the camera (Section 8).
3. **Part clears** (the `elif sensorState:` branch, i.e. the sensor reads inactive again): first it un-flags `sensorTriggered` and turns the conveyor-shorten pin back off. Then — this is the actual OK/NG hold mechanism — line (B) checks `result_active`. That flag is set to `True` by `set_ok()/set_ng()` (above) whenever a camera result arrives, and is *only* cleared back to off here, when the sensor confirms the part has physically left the station. Between those two moments, `OK_PIN/NG_PIN` stay exactly as `set_ok()/set_ng()` left them — there is no timer involved. That is the entire "hold" behavior: it is a direct consequence of *when this method decides to reset the pins*, not a separate pulse-width mechanism.
4. **Why it matters if you are modifying this file:** if a result arrives before the previous part has cleared the sensor (i.e. `result_active` is still `True` from the prior cycle), the new `set_ok()/set_ng()` call simply overwrites the pin state and `result_active` stays `True` — there is no queueing or per-cycle isolation here. The design assumes one part is under inspection at a time.

Walkthrough 2 — Camera protocol parsing (`serial_receiver.py`, `SerialReceiver._onLine()`)

```
def _onLine(self, data):  
  
    # (A)  
  
    if not self.isReady:  
        self.isReady = True  
        self.switchReady.emit(True)  
        return  
  
    # (B)  
  
    if time.monotonic() < self.suppressResultsUntil:
```

```

        return

# (C)

if data == "1;":

    part = str(self.currentScheme)

    self.resultReceived.emit(part, "OK")

# (D)

elif data.upper() == "NG":

    part = str(self.currentScheme)

    self.resultReceived.emit(part, "NG")

# (E)

else:

    parts = data.split(",")

    if len(parts) == 2:

        part = parts[0].strip()

        status = parts[1].strip().upper()

        if part in ["1","2","3","4"] and status in ["OK","NG"]:

            self.resultReceived.emit(part, status)

```

1. Every line the camera sends over serial arrives here, already decoded and stripped by the background `SerialWorker` thread (above). This method runs on the GUI thread.
2. **(A) is a one-shot gate, not a content check.** `isReady` is set `False` the instant `selectScheme()` sends a switch command. So the very next line that arrives — *whatever its content is* — is consumed here as that switch's own acknowledgement/echo and never reaches the OK/NG logic below. This is why switching schemes right before a real inspection result arrives cannot mis-count that result as an ack: the flag is already `True` again well before that.

3. **(B) is a time-based second gate**, independent of (A): for 0.5s after a scheme switch (`SWITCH_COOLDOWN_SECONDS`), any further line is silently dropped. This exists for stray echoes that arrive after the ack itself but are still switch-related noise, not a real inspection.
4. **(C) and (D) are the only two "real camera" formats currently in use**. Note line (C) is an exact match on the literal string `"1;"` — not `"OK"`. A previous version treated the text `"OK"` as a result, which caused false OK counts because the switch ack itself sometimes contained that word; that is why gate (A) exists and why (C) checks for `"1;"` specifically instead.
5. **(E) is legacy fallback** for an older two-value wire format (`"1, OK"`) — it validates both the part number and the status strictly before emitting anything, so a malformed or unrelated line is silently ignored rather than crashing or emitting garbage.
6. **Why it matters if you are modifying this file:** gates (A) and (B) run *before* the format checks, on every single line unconditionally. Adding a new camera response format means adding a branch after (E), not before — anything added before (A)/(B) would bypass the switch-noise filtering this method exists to provide.

7. Application Execution Flow

This section traces exactly what happens between running `python main.py` (or launching `dist/main`) and the app being ready for the first inspection.

Startup Sequence

1. `setup_error_log()` runs first, before anything else. It creates a `RotatingFileHandler` writing to `logs/app.log` next to the app (2MB per file, 3 backups kept), then redirects `sys.stdout` and `sys.stderr` through `_StreamToLog` so every existing `print()` call anywhere in the codebase is captured to the log file as well as the terminal, and installs a global `sys.excepthook` so an uncaught exception is logged instead of silently crashing an unattended, autostarted app.
2. `QT_QUICK_CONTROLS_STYLE` is set to `Basic` in the environment — must happen before `QGuiApplication` is constructed, which is why it is the very next line (the lightest Qt Quick Controls style available, chosen to cut Qt's startup overhead on the Pi 4).
3. `QGuiApplication` and `QQmlApplicationEngine` are created, and `load_config()` reads `config.json` (falling back to `/dev/ttyUSB0 @ 9600` if the file is missing or invalid).
4. The four core objects are instantiated in this order: `ProductionModel()`, `SerialReceiver(port, baudrate)`, `GPIOController()`, `LogModel()`. None of them depend on each other directly — `Backend` is what wires them together (Section 6).
5. `LogModel` is seeded with the last 5 entries already present in `counts.json` (via `productionModel.logs[-5:]`), so the on-screen Log panel is not empty immediately after a restart.
6. `Backend(productionModel, serialReceiver, gpiocontroller, logModel)` is created — this is the hub described in Section 6.
7. `serialReceiver.connectSerial()` starts the background serial thread and its watchdog timer.
8. `app.aboutToQuit` is connected to `serialReceiver.disconnectSerial` and `gpiocontroller.cleanup`, so the serial thread is stopped and GPIO lines are released cleanly on exit — without this, Qt aborts the process because a background thread is still alive when the app object is destroyed.
9. `backend`, `gpiocontroller`, `productionModel`, and `logModel` are registered as QML

context properties (by exactly those names), which is what lets QML files reference `backend.xxx`, `productionModel.xxx`, etc. directly with no import or wiring on the QML side.

`10.tech/Main.qml` is loaded as the application's entry point via `engine.loadFromModule("tech", "Main")`, and `app.exec()` starts the Qt event loop. If no root object loaded, the process exits with status `-1`.

Main Functions Involved

Function	Called	Purpose
<code>setup_error_log()</code>	Step 1, once	Wire up logging before anything can fail silently
<code>load_config()</code>	Step 3, once	Resolve the serial port/ baud rate to use for this run
<code>if __name__ == "__main__": block</code>	Process entry	Everything above — construct, wire, connect, load QML, run the event loop

8. UI-to-Code Mapping & Data Flow

UI Control → Code Mapping

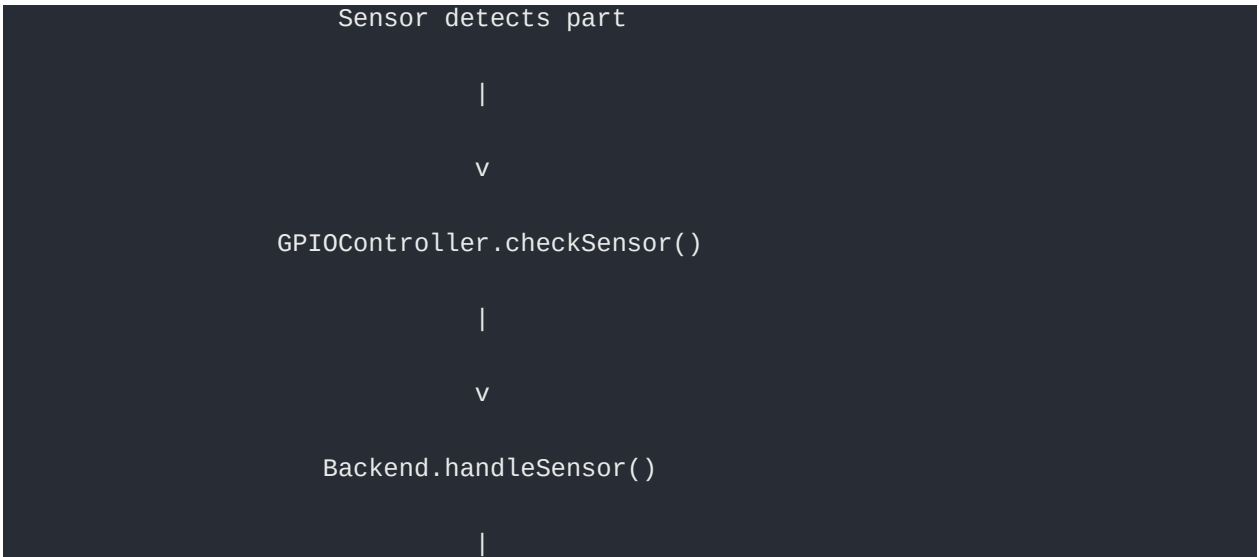
UI Control	QML File	Call Made	Effect
"Part 1"—"Part 4" buttons	Main.qml	<code>stackView.push("PartN.qml")</code>	Navigates to that part's screen; no backend call yet
START	Part1-4.qml	<code>gpiocontroller.conveyor_on()</code>	Drives <code>CONVEYOR_PIN</code> active. Enabled only when <code>backend.schemeReady</code> is true. Direct call — bypasses Backend.
STOP	Part1-4.qml	<code>gpiocontroller.conveyor_off()</code> , then <code>stackView.pop()</code>	Conveyor off, returns to Home
BACK	Part1-4.qml	<code>gpiocontroller.conveyor_off()</code> , then <code>stackView.pop()</code>	Same effect as STOP — also stops the conveyor before leaving the screen
RESET	Part1-4.qml	<code>productionModel.resetPart(N)</code>	Captures a new baseline; the on-screen tiles drop to 0, lifetime totals and the report are unaffected
Report	Main.qml	<code>stackView.push("ProductionReport.qml")</code>	Opens the report screen
Power → Shutdown/Reboot	Main.qml	<code>backend.shutdownSystem()</code> / <code>rebootSystem()</code>	Runs after a confirm dialog; both need the passwordless sudo rule (Section 18). A

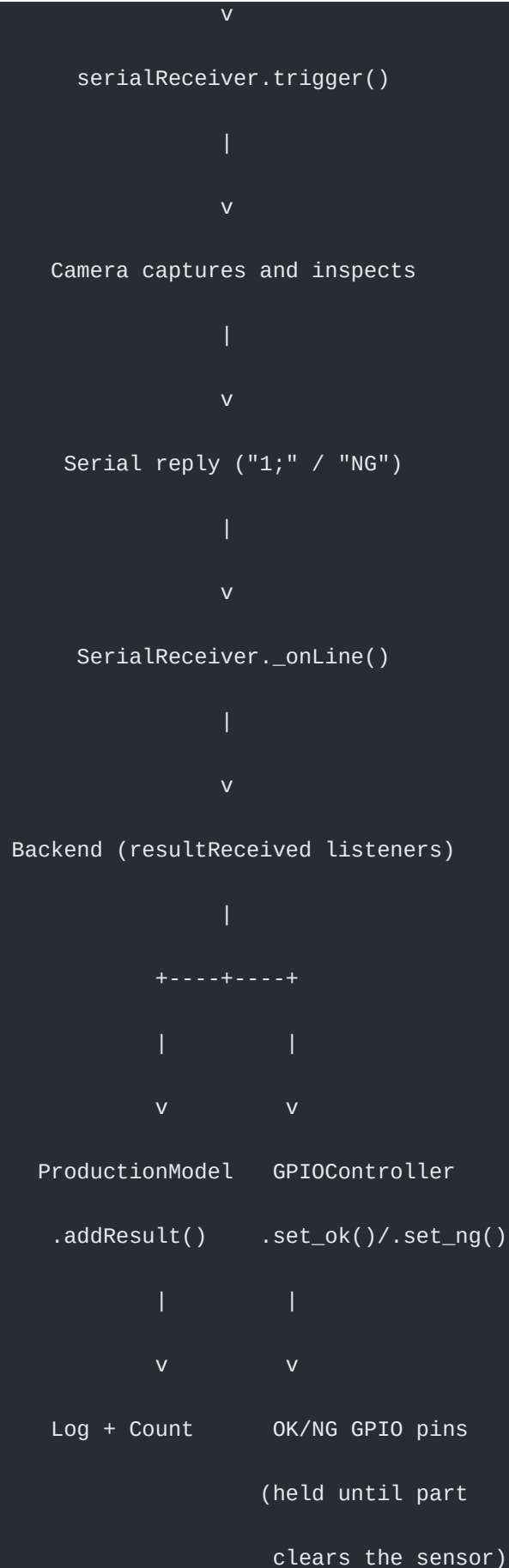
third option, Restart, exists in `backend.py` (`restartApp()`) but its menu item is currently commented out in `Main.qml` — not reachable from the running app (Section 6).

Full Reset	<code>ProductionReport.qml</code>	<code>productionModel.resetData(),</code> <code>logModel.clear()</code>	Wipes every part's counts, baselines, and the log history
<i>(physical sensor, not UI)</i>	—	<code>gpioController.sensorDetected</code> → <code>Backend.handleSensor()</code> → <code>serialReceiver.trigger()</code>	Automatically triggers the camera when a part is detected
<i>(camera result, not UI)</i>	—	<code>serialReceiver.resultReceived</code> → <code>Backend.addResult()</code> + <code>handleGPIOResult()</code>	Updates counts/log and drives the OK/NG GPIO pins

Data Flow: One Full Inspection Cycle

The component-level path a single result travels, from the physical sensor through to the two places it ends up (the persisted log/count and the physical GPIO signal):





Numbered step by step, with the reasoning behind each hop:

1. A part physically arrives under the sensor → `GPIOController`'s 50ms polling timer notices `SENSOR_PIN` go active and emits `sensorDetected`.
2. `Backend.handleSensor()` receives that signal and calls `serialReceiver.trigger()`, which queues a `b"T"` command for the camera.
3. The camera captures, inspects, and writes its result back over serial (e.g. `1;` for OK, or `NG`). The background `SerialWorker` thread reads the line and emits `lineReceived`.
4. `SerialReceiver._onLine()` parses it (filtering out scheme-switch acks/cooldown — see Section 10) and emits `resultReceived(part, status)`.
5. `Backend` has two listeners on that same signal: `addResult()` records it into `ProductionModel` (counters + persisted to `counts.json`) and into `LogModel` (for the on-screen log panel), while `handleGPIOResult()` drives the physical `OK_PIN/NG_PIN` via `GPIOController.set_ok()/set_ng()`.
6. The OK/NG GPIO output stays in that state until the part physically clears the sensor, at which point `GPIOController.checkSensor()` resets both pins back off (Section 10).
7. `ProductionModel.dataChanged` firing (from step 5) causes `Backend.emitSignals()` to re-emit every part's OK/NG/Total signal, which is what makes the counters on the currently open Part page update live.

9. UI Walkthrough — QML Screens

Main.qml — App Shell / Home Screen

- An `ApplicationWindow` that starts in real OS-level fullscreen (`Window.FullScreen`) — this is the kiosk behavior described in Section 15.
- `scaleFactor` is computed as `min(width / 1280, height / 720)` and multiplies every hardcoded size/font/margin in this file and every pushed page, so the same UI scales proportionally whether it lands on a small panel or a dev monitor, using 1280×720 as the design baseline.
- **F11** toggles fullscreen ↔ windowed; **Escape** drops out of fullscreen the same way — the documented admin escape hatch, not shown to the client.
- A `StackView` is the navigation container for the whole app; the Home page is defined inline as a `Component` and pushed as its `initialItem`.
- Home page contents: logo and company name, four "Part N" buttons, a "Report" button, and a power button opening a Shutdown / Reboot menu that routes through one shared confirmation `Dialog` (see Section 8's mapping table for the exact calls). A Restart option exists in the QML source but is currently commented out (see the note in Section 8).
- **Styling gotcha worth knowing:** Qt Quick `Menu/Dialog` popups do not inherit the window's `palette.text` override, so without explicit colors on every `MenuItem/Label` inside them, their text renders dark-on-dark. Both the power menu and the confirm dialog set colors explicitly for this reason — keep doing so if more popups are added.

Part1.qml – Part4.qml — Per-Part Screens

The four files are near-identical, differing only in which part number they pass to the backend and production model calls.

- On load, `backend.selectScheme(N)` switches the camera to that part's inspection program, then `refreshCounts()` loads the "since reset" counts via `productionModel.getOKSinceReset(N)` etc.
- A 1-second `Timer` drives the live date/time header.
- A `Connections` block on `productionModel.dataChanged` re-runs `refreshCounts()` whenever a new result is recorded.

- Three colored tiles (green OK, red NG, blue TOTAL) are bound to the "since reset" counts.
- Embeds the reusable `Logs` component (below) plus a "LAST RESULT" tile bound to `logModel.latestStatus / latestTime`.

tech/contents/Logs.qml — Reusable Log Panel

A connection-status badge (red "NOT CONNECTED" / orange "SWITCHING..." / green "READY", driven directly off `backend.connected` and `backend.schemeReady`) above a `ListView` bound to the `logModel` context property, rendering its `time / event / status` roles as a striped table with OK in green and NG in red. Embedded on every Part page.

ProductionReport.qml

Builds a table from `productionModel.getTotal/getOK/getNG` for all four parts (`refreshPartsTable()`, re-run whenever `productionModel.dataChanged` fires) alongside grand totals from `productionModel.totalParts/totalOK/totalNG`. Its Reset control calls `productionModel.resetData()` and `logModel.clear()` — the full-wipe reset, distinct from each Part page's own baseline-only Reset button.

JobReport.qml — Unreachable / Incomplete Screen

Known issue, documented for anyone touching this code: this screen binds its date-filter search and its list to a `jobReportModel` context property, but no such property is ever registered in `main.py` (only `backend`, `gpiocontroller`, `productionModel`, and `logModel` are). There is also no `stackView.push("JobReport.qml")` call anywhere else in the codebase, so this page is not reachable from the running app at all. Treat it as leftover/incomplete work, not a bug in something users can currently hit — if a Job Report screen is wanted, it needs a model wired up and a button somewhere to open it.

10. Hardware Reference

Target Hardware

This system is built and deployed for the **Raspberry Pi 4 Model B** — the only Pi 4 variant that exists (there is no Model A for Pi 4).

Spec	Detail
SoC	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit @ 1.5GHz
RAM	4GB LPDDR4-3200 (this station's deployed variant — the board also ships in 1GB / 2GB / 8GB)
Boot storage	microSD card (this station's deployed medium)
GPIO	Standard 40-pin header, BCM numbering — what the pin map below targets
USB	2× USB 3.0, 2× USB 2.0 — the camera's USB-to-serial adapter (below) uses one of these
Video	2× micro-HDMI (dual 4K capable) — drives the display the kiosk UI (Section 9) renders to
Networking	Gigabit Ethernet; 2.4GHz/5GHz 802.11ac Wi-Fi; Bluetooth 5.0
Power	USB-C, 5V/3A recommended

An SD card is the most common cold-start bottleneck on a Pi 4 — if the app's cold-start time (Section 4) ever needs to come down, moving ~/tech to a USB SSD is the single biggest lever available on this station's current hardware.

GPIO Pin Map (BCM numbering)

Pin No.	Name	Direction	Purpose
17	SENSOR_PIN	Input	Part-present sensor
22	OK_PIN	Output	OK signal to the line controller

23	<code>NG_PIN</code>	Output	NG signal to the line controller
24	<code>CONVEYOR_SHORTEN_PIN</code>	Output	Shortens the conveyor cycle
27	<code>CONVEYOR_PIN</code>	Output	Conveyor run/stop

Design note: the OK/NG output pins hold their state until the sensor clears, rather than firing a fixed-width pulse. This is intentional and should not be changed without understanding why — it matches the timing the line controller expects. The exact mechanism is `GPIOController.checkSensor()`, annotated in Section 6.

Physical Wiring

- **`SENSOR_PIN (17)` — input.** Wired directly from the physical part-presence sensor's output to GPIO17.
- **All four output pins — `OK_PIN (22)`, `NG_PIN (23)`, `CONVEYOR_SHORTEN_PIN (24)`, `CONVEYOR_PIN (27)` — each drive a relay.** The GPIO output switches the relay's coil side; the relay's switched contacts are what actually connect to the line controller/conveyor equipment. This electrically isolates the Pi's 3.3V logic from whatever voltage that equipment runs on.

Not yet documented here: the specific relay module/board in use, its coil voltage and contact rating, and which physical terminal on the line controller each relay's output lands on. Record that here once available — this reference currently covers the GPIO side only, not the electrical side beyond "goes through a relay."

How the Camera Communication Works (Plain-Language Overview)

For anyone new to this project: the Pi and the camera are not on a network together, and there is no app or API involved. They are joined by a single serial cable (through a USB-to-serial adapter), and they simply take turns sending short lines of plain text back and forth over it — the same basic idea as two people passing short notes, one at a time, at an agreed reading speed (the baud rate).

- The Pi is the one that starts each exchange — it sends a short command like `T` ("take a picture now") or `S PART1` ("switch to part 1's inspection program").
- The camera does its own work (capturing and inspecting the part) independently, then

writes a short reply back on the same wire — `1;` for a pass, `NG` for a fail.

- The Pi is always listening for that reply; when one arrives, that is what starts the counting/logging/GPIO steps in Section 8's data-flow walkthrough.

How to build/set up this link:

1. Connect the camera's serial output to the Pi using a USB-to-serial (RS232-to-USB) adapter cable — the camera side wires into the adapter, the adapter's USB end plugs into the Pi.
2. Confirm the Pi sees it as a device: `ls /dev/ttyUSB*`. It will typically show up as `/dev/ttyUSB0`.
3. Both sides must agree on the same speed (baud rate) — this project uses **9600**, matching whatever the camera unit is configured to use on its own serial output. If the camera's own settings are changed, `baudrate` in `config.json` (Section 11) must be changed to match, or nothing readable will arrive on either end.
4. That is the entire physical/setup side — once the port shows up and the baud rate matches, `serial_receiver.py` (Section 6) handles everything else in software.

Camera Serial Protocol (Exact Reference)

- Port and baud rate are read from `config.json` (`serial_port`, `baudrate` — default `/dev/ttyUSB0 @ 9600`).
- Camera sends `1;` → treated as an OK result.
- Camera sends `NG` → treated as an NG result.
- Literal text `"OK"` is deliberately ignored as a result — it was previously causing false OK counts right after a scheme switch (ack/echo, not a real inspection result).
- Multi-part format: `1,OK / 2,NG` (part number, result).
- A 0.5s cooldown window after a scheme switch ignores incoming results, to avoid counting a switch acknowledgement as a real inspection.
- A watchdog reconnects the serial connection automatically if no heartbeat arrives for 30s.

See Section 6 for the full `backend.py/gpio_controller.py/serial_receiver.py` function reference and the annotated walkthroughs of the two logic blocks above.

11. Configuration Reference

- `config.json` — `serial_port` (e.g. `/dev/ttyUSB0`), `baudrate` (e.g. `9600`)
- `counts.json` — persisted per-part production counts, baselines, and log history; survives app restarts

Part II — Deployment & Operations Guide

[OS Install](#) · [Build](#) · [Kiosk Mode](#) · [Boot Customization](#) · [Troubleshooting](#)

12. Raspberry Pi OS Installation & System Prerequisites

Part II from here on (Sections 13–21) assumes a Raspberry Pi already running Raspberry Pi OS with these prerequisites in place. This section is what gets a completely bare Raspberry Pi 4 — nothing installed, no SD card written — to that starting point.

1. Download and run **Raspberry Pi Imager** from `raspberrypi.com/software` on any PC, then write an SD card (or USB SSD):
 - Choose **Device**: Raspberry Pi 4
 - Choose **Operating System**: Raspberry Pi OS (64-bit), with desktop
 - Choose **Storage**: the target SD card / SSD
2. Before writing, open the Imager's advanced options (the gear icon / `Ctrl+Shift+X`) and pre-configure:
 - Hostname (see Section 16 for why a non-default hostname matters for a client-facing kiosk)
 - Username and password for the default account
 - Wi-Fi SSID/password, if not using Ethernet
 - Enable SSH (convenient for headless setup/debugging later)

Pre-configuring here avoids the first-boot setup wizard entirely on later boots.

3. Insert the card into the Pi and power it on. If SSH was enabled and the network is reachable, connect from another machine:

```
ssh <username>@<hostname>.local
```

Otherwise continue directly on the Pi's own screen/keyboard.

4. If the advanced options in step 2 were not used, complete the on-screen first-boot wizard: country/language/timezone, Wi-Fi, and setting a password.
5. Update the OS before installing anything else:

```
sudo apt update

sudo apt full-upgrade -y

sudo reboot
```

6. Install the system-level packages the rest of Part II depends on. None of these are Python packages — they are what has to already be present on the OS before `pip install -r`

`requirements.txt` (Section 13) can succeed:

```
sudo apt install -y python3-venv python3-pip python3-dev build-essential
libgpod-dev git
```

Package	Why it is needed
<code>python3-venv</code>	Provides the <code>venv</code> module — without it, <code>python3 -m venv venv</code> (Section 13) fails
<code>python3-pip</code>	<code>pip</code> itself, for installing the Python packages in Section 4
<code>python3-dev</code> , <code>build-essential</code>	Headers and a compiler, needed if <code>pip install gpod</code> has to build the package from source because no prebuilt wheel matches this Pi's exact Python version/architecture
<code>libgpod-dev</code>	The system GPIO character-device library the Python <code>gpod</code> package links against when compiling
<code>git</code>	Optional — only needed if pulling the project via <code>git clone</code> instead of copying it over by USB drive (Section 13, step 1)

7. Confirm the GPIO character device is present (it is by default on Raspberry Pi OS — this is just a sanity check before Section 13/14's GPIO group setup):

```
ls /dev/gpiochip*
```

Should list at least `/dev/gpiochip0`.

Result: a Raspberry Pi with Raspberry Pi OS installed and updated, network access configured, and every system-level package Part II's Python setup (Section 13 onward) needs already in place. Continue to Section 13.

13. Setup and Run (Development)

1. Copy the project to the Pi via USB drive, into `~/tech`. The source lives on the Ubuntu development workstation (Section 1) at the project root — this step moves it across to the Pi.

On the Ubuntu development machine (where the code is written):

1. Insert a USB drive. Confirm it appears and find its device name:

```
lsblk
```

On Ubuntu with a desktop session it is usually auto-mounted under

`/media/$USER/<label>`; if not, mount it with `udisksctl mount -b /dev/<device>`.

2. Copy the project onto the drive, leaving out build artifacts and IDE-local folders that the Pi does not need and will regenerate itself:

```
rsync -av \

  --exclude='__pycache__' \

  --exclude='build' \

  --exclude='dist' \

  --exclude='.qtcreator' \

  --exclude='.claude' \

  --exclude='diagnostics' \

  --exclude='logs' \

  --exclude='counts.json' \

~/DEV_ENV/tech /media/$USER/<label>/
```

No trailing slash on `tech` and no folder name on the destination — the project directory is already called `tech`, so `rsync` creates `/media/$USER/<label>/tech/` by itself, using that existing name rather than it being typed twice. `counts.json` is excluded deliberately — it holds this dev machine's own test/production counts. `ProductionModel.loadData()` creates a fresh, empty one automatically on the

Pi if it is missing (Section 6), which is what a new station should start from.

3. Unmount the drive cleanly before removing it:

```
udisksctl unmount -b /dev/<device>
```

(or use the file manager's eject button).

On the Raspberry Pi:

4. Insert the same USB drive. It should auto-mount under `/media/pi/<label>` (check with `lsblk` if not).

5. Copy it into place:

```
cp -r /media/pi/<label>/tech ~/
```

Again, the source folder is already named `tech`, so copying it straight into the home directory produces `~/tech` on its own — no separate `mkdir` or renaming needed.

6. Unmount and remove the drive the same way as on the dev machine:

```
udisksctl unmount -b /dev/<device>
```

2. Open a terminal and go into the project folder:

```
cd ~/tech
```

3. Create a virtual environment:

```
python3 -m venv venv
```

4. Activate the virtual environment:

```
source venv/bin/activate
```

Once activated, it stays active for the rest of that terminal session.

5. Install dependencies (installs PySide6, pyserial, and gpiozero together):

```
pip install -r requirements.txt
```

6. Add yourself to the `gpio`/`dialout` groups (only needed once per Pi):

```
sudo usermod -aG gpio,dialout $USER
```

Log out/in (or reboot) before running — this will not take effect in the current session.

7. Set the camera's serial port in `config.json`:

```
ls /dev/ttyUSB*
```

```
nano config.json
```

8. Run the app:

```
python main.py
```

14. App Creation (Build Standalone Executable)

1. Go into the project folder and activate the virtual environment (skip if still active from Section 13):

```
cd ~/tech  
  
source venv/bin/activate
```

2. Install PyInstaller:

```
pip install pyinstaller
```

3. Build the standalone executable:

```
pyinstaller --clean --noconfirm main.spec
```

Produces `dist/main` — a single self-contained binary that does not need the venv at runtime.

4. Copy the config next to the built binary:

```
cp config.json dist/config.json  
  
deactivate
```

5. Confirm GPIO/serial groups (only needed if not already done on this Pi):

```
sudo usermod -aG gpio,dialout $USER
```

If you just ran that, log out/in (or reboot) before testing.

6. Set the camera's serial port for the built binary:

```
ls /dev/ttyUSB*  
  
nano ~/tech/dist/config.json
```

7. Test the built binary:

```
~/tech/dist/main
```

Confirm the window opens, serial connects, and GPIO OK/NG behaves correctly.

8. Install the desktop launcher:

```
cd ~/tech
```

```
sed -i "s|/home/pi/tech|$HOME/tech|g" deploy/production-app.desktop

mkdir -p ~/.local/share/applications

cp deploy/production-app.desktop ~/.local/share/applications/

cp deploy/production-app.desktop ~/Desktop/

chmod +x ~/Desktop/production-app.desktop
```

9. Use it: double-click **Production Line Monitor** on the Desktop (or find it in the applications menu).

15. Autostart the Application (Kiosk Mode)

Objective: boot directly into the application, skipping the desktop and login screen entirely.

1. Enable desktop auto-login:

```
sudo raspi-config
```

Navigate to: `System Options` → `Boot / Auto Login` → `Desktop Autologin`

2. Auto-start the application when the desktop session starts:

```
mkdir -p ~/.config/autostart
```

```
cp ~/.local/share/applications/production-app.desktop ~/.config/autostart/
```

3. Restart the Raspberry Pi to confirm:

```
sudo reboot
```

4. Verify autostart is actually firing directly (LXDE/X11). Confirm the session type rather than assuming it:

```
echo $XDG_CURRENT_DESKTOP
```

```
echo $XDG_SESSION_TYPE
```

```
ps -e | grep -E 'labwc|lxsession|lxpanel'
```

On this Pi that returns `LXDE / X11`, confirming `lxsession` is the running session manager and the newer Wayland/labwc stack is not in use. XDG autostart (step 2 above) only launches the app *after* the desktop session has already started, so briefly seeing the desktop before the app appears can happen even when it is "working" — confirm the actual timing:

```
pgrep -af dist/main
```

Reboot, then re-run that command. If the desktop is visible for more than a brief moment before the app appears, launch the app directly from `lxsession`'s own autostart file instead — this runs at session start regardless of XDG timing, and `@`-prefixed lines are automatically respawned if the app exits, which suits an app meant to stay running. **The system autostart file is not always at the commonly documented `LXDE-pi` path** — locate it rather than assuming:

```
find /etc/xdg/lxsession -name autostart -type f
```

On this Pi that resolved to `/etc/xdg/lxsession/rpd-x/autostart` (the profile folder is named `rpd-x` on this Raspberry Pi OS image, not `LXDE-pi`). Use whatever path that command reports in place of `rpd-x` below if it differs:

```
mkdir -p ~/.config/lxsession/rpd-x  
  
cp /etc/xdg/lxsession/rpd-x/autostart ~/.config/lxsession/rpd-x/autostart  
  
echo "@$HOME/tech/dist/main" >> ~/.config/lxsession/rpd-x/autostart
```

Reboot again and re-check with `pgrep -af dist/main` — the app should now come up without the desktop showing first.

Fullscreen escape: the app opens in true fullscreen and covers the taskbar/panel — this is what the client sees. **F11** drops it back to a normal window (title bar, taskbar/panel visible, file manager reachable); **F11** or **Esc** returns to fullscreen. This is an administrative access method and should not be shared with the client as a documented feature.

16. Boot Screen Customization

Prerequisite: Section 15 (Autostart / Kiosk Mode) must already be configured. Without it, the boot screen will still go black, but it will lead to the normal desktop or a login prompt instead of the application.

Objective: display a plain black screen during boot by removing the Raspberry Pi branding, rainbow splash screen, and system boot messages before the application launches.

1. Disable the Raspberry Pi splash screen:

```
sudo raspi-config
```

Navigate to: System Options → Splash Screen → No

2. Configure the boot command line:

```
sudo nano /boot/firmware/cmdline.txt
```

The file contains a single line. Append the following parameters to the existing line (do not create a new line):

```
splash quiet loglevel=0 logo.nologo vt.global_cursor_default=0  
systemd.show_status=0
```

3. Create a blank Plymouth theme:

```
sudo mkdir -p /usr/share/plymouth/themes/blank  
  
sudo tee /usr/share/plymouth/themes/blank/blank.plymouth > /dev/null <<'EOF'  
  
[Plymouth Theme]  
  
Name=Blank  
  
Description=Solid black screen, no branding  
  
ModuleName=script  
  
[script]  
  
ImageDir=/usr/share/plymouth/themes/blank  
  
ScriptFile=/usr/share/plymouth/themes/blank/blank.script
```

```
EOF
```

4. Create the blank theme script:

```
sudo tee /usr/share/plymouth/themes/blank/blank.script > /dev/null <<'EOF'

Window.SetBackgroundTopColor(0, 0, 0);

Window.SetBackgroundBottomColor(0, 0, 0);

EOF
```

5. Set the blank Plymouth theme as default:

```
sudo plymouth-set-default-theme -R blank
```

The `-R` option rebuilds the initramfs so the theme is applied from early boot.

6. Verify the boot configuration:

```
sudo nano /boot/firmware/cmdline.txt
```

Confirm the parameters from step 2 are present on the same single line.

7. Change the default hostname (optional, recommended — or pre-set it in Raspberry Pi Imager per Section 12, step 2). The default `raspberrypi` hostname can surface in network device lists, mDNS/.local discovery, or an SSH prompt if the client ever gets a terminal:

```
sudo raspi-config
```

Navigate to: `System Options` → `Hostname` → set something client-neutral, e.g. `station01`.

8. Restart the Raspberry Pi:

```
sudo reboot
```

Expected result: after rebooting, the screen goes solid black from very early boot straight through to the application — no rainbow splash, no scrolling boot text, no login prompt, no branding.

17. Hide the Taskbar / Panel

The fullscreen app only covers the panel while running — escape with F11 and it is back. This disables it outright so it never shows, even when escaped to the desktop. These are the exact steps verified on this Pi — the commonly documented LXDE-pi profile path did not exist here, so each step below confirms the real value first rather than assuming it.

1. Confirm the current desktop/session:

```
echo $XDG_CURRENT_DESKTOP  
  
echo $XDG_SESSION_TYPE
```

This returned LXDE and X11. Then:

```
ps -e | grep -E 'labwc|lxsession|lxpanel'
```

This confirmed lxsession/lxpanel-pi were running and labwc (the newer Wayland session some Raspberry Pi OS images default to) was not.

2. Find the correct LXDE autostart file. The commonly documented path, /etc/xdg/lxsession/LXDE-pi/autostart, did not exist on this Pi. Search instead of assuming the path:

```
find /etc/xdg/lxsession -name autostart -type f
```

This found the real one: /etc/xdg/lxsession/rpd-x/autostart.

3. Check the original autostart file:

```
cat /etc/xdg/lxsession/rpd-x/autostart
```

It contained:

```
@lxpanel-pi  
  
@pcmanfm-pi  
  
@xscreensaver -no-splash
```

@lxpanel-pi is the panel startup command — note the -pi suffix, which differs from the generic lxpanel name used on some other Raspberry Pi OS builds.

4. Create a personal LXDE configuration folder, so the system file under /etc/xdg is never modified directly:

```
mkdir -p ~/.config/lxsession/rpd-x
```

5. Copy the system autostart file into it:

```
cp /etc/xdg/lxsession/rpd-x/autostart ~/.config/lxsession/rpd-x/autostart
```

6. Comment out the panel line:

```
sed -i 's/^@lxpanel-pi/#@lxpanel-pi/' ~/.config/lxsession/rpd-x/autostart
```

This turns `@lxpanel-pi` into `#@lxpanel-pi` — the leading `#` comments the line out so `lxsession` skips it.

7. Verify the change:

```
cat ~/.config/lxsession/rpd-x/autostart
```

Confirms:

```
#@lxpanel-pi
@pcmanfm-pi
@xscreensaver -no-splash
```

Only the panel line is disabled — the file manager and screensaver lines are untouched.

8. Reboot to apply:

```
sudo reboot
```

To check it worked:

```
pgrep -a lxpanel-pi
```

No output means it is not running.

To restore the panel:

```
sed -i 's/^#@lxpanel-pi/@lxpanel-pi/' ~/.config/lxsession/rpd-x/autostart
```

Then either reboot, or start the panel immediately without one:

```
lxpanel-pi &
```

Why rpd-x, specifically: that is the actual LXDE session configuration folder name on this Raspberry Pi's OS image — it is not a typo for the more commonly documented LXDE - `pi`, and the two should not be mixed. This step did not modify `labwc`, `GPIO`, or serial configuration — only the panel's autostart line, and only in the user's own config copy, never the system file under `/etc/xdg`.

Trade-off: the panel is also where the Raspberry Pi start menu lives, so disabling it removes that as an admin way back in. Right-click-on-desktop (via `pcmanfm`) and SSH still work for getting back in — confirm one of those suits before relying on this.

18. Passwordless Shutdown / Reboot Buttons

The Home screen's Shutdown/Reboot buttons call `sudo shutdown -h now` and `sudo reboot` (see Section 6, `backend.py`). The app has no terminal to type a sudo password into, so without this step those two buttons do nothing except log a permission error to `app.log`. The app-level Restart action (`restartApp()`) would not need this — it just relaunches the app binary, no sudo involved — but it is not currently reachable from the UI (Section 6).

```
echo "$USER ALL=(ALL) NOPASSWD: /sbin/shutdown -h now, /sbin/reboot" | sudo tee
/etc/sudoers.d/010-tech-power

sudo chmod 440 /etc/sudoers.d/010-tech-power

sudo visudo -c
```

`visudo -c` checks the new file's syntax before it takes effect — fix and re-run if it reports an error. This is scoped to exactly those two commands for this user only, not a blanket sudo grant.

Test from the app (not a terminal, so the passwordless rule is actually what runs it): tap Reboot, confirm the dialog, confirm the Pi reboots.

19. Error Log & Troubleshooting

Every status/error message the app prints (GPIO init, serial connect/disconnect, camera results, crashes) is written to:

```
~/tech/dist/logs/app.log
```

It auto-rotates at 2MB and keeps 3 backups (`app.log.1`, `app.log.2`, `app.log.3`), so it is safe to leave running long-term. Check this file first when something goes wrong and nobody was watching a terminal at the time — e.g. after autostart/boot.

Finding Errors in the Log

The log mixes routine status lines (GPIO init, serial connect, every camera result) with the actual errors — do not read it top to bottom. Search it instead:

- Errors only, across the current log and all rotated backups:

```
grep -i "error" ~/tech/dist/logs/app.log*
```

- Watch it live while reproducing a problem (e.g. right after autostart):

```
tail -f ~/tech/dist/logs/app.log
```

- Every log line is timestamped `%(asctime)s` `[% (levelname)s]` `%(message)s`, set up in `setup_error_log()`, Section 7) and uncaught exceptions are logged with a full traceback — so a crash always leaves an `[ERROR]` line with the exact Python stack trace in this file, not just a silent exit.

Common issues

- **Serial port not found** — check `ls /dev/ttyUSB*` and that `config.json` / `dist/config.json` point to the right device.
- **`pip install gpiod` fails to build** — missing system prerequisites; see Section 12, step 6 (`python3-dev`, `build-essential`, `libgpiod-dev`).
- **GPIO permission errors** — confirm the user is in the `gpio` group (Section 13/14, step "Confirm GPIO/serial groups") and has logged out/in since.
- **Autostart not firing / desktop shows first** — see Section 15, step 4 (LXDE verification and fallback).
- **Shutdown/Reboot buttons do nothing** — see Section 18 (passwordless sudo rule not

yet applied).

20. Updating an Existing Install

Use this when the Pi already has the app fully set up (Sections 13–17 already done) and newer code changes just need to be picked up.

```
cd ~/tech

# copy/pull the updated source files over the existing ones here

# (main.py, tech/Main.qml, etc.) - do NOT overwrite dist/config.json,
# it holds this Pi's tuned serial port

source venv/bin/activate

pyinstaller --clean --noconfirm main.spec

deactivate
```

The root `config.json` is only copied to `dist/config.json` once (Section 14, step 4) — if `dist/config.json` already exists on this Pi, leave it alone so the tuned serial port setting is not lost.

Autostart (Section 15) does not need to be redone — it launches `~/tech/dist/main`, which the rebuild just replaced in place. Reboot to run the updated app.

21. Complete Project Working Flow — Build to Cleanup

Every earlier section covers one phase in isolation. This section reads top to bottom as one continuous story — from a bare Raspberry Pi, through daily use, to the app shutting itself down cleanly — so someone with no prior context can follow the whole project in one pass and know exactly where to go for the detail behind any step.

```
+-----+
| PHASE 1: BUILD (once per Pi)           Sections 12-18 |
| Flash OS, install prerequisites, build the binary,    |
| wire up kiosk mode                                |
+-----+
|
| v
+-----+
| PHASE 2: BOOT (every power-on)         Sections 15-16, 7|
| Black screen -> lxsession launches dist/main ->      |
| main.py startup sequence -> fullscreen Home screen  |
+-----+
|
| v
+-----+
| PHASE 3: WORKING FLOW (all day)         Sections 8-9|
| Select part, tap START -> inspection loop runs on its |
| own (sensor -> camera -> result -> GPIO -> log) ->   |
| operator can Reset, view Report, switch parts, repeat |
+-----+
```

```
|
v (power button tapped, or normal quit)

+-----+
| PHASE 4: CLEANUP (every shutdown/reboot)    Sections 6, 9|
| app.aboutToQuit fires -> gpiocontroller.cleanup() +      |
| serialReceiver.disconnectSerial() -> OS shutdown/reboot  |
| actually happens                                     |
+-----+
```

Phase 1 — Build (once per Pi)

Starting from a completely bare Raspberry Pi 4:

1. Flash Raspberry Pi OS and install the system prerequisites (Section 12).
2. Copy the project over, create the venv, install Python dependencies, confirm it runs from source (Section 13).
3. Build the standalone binary with PyInstaller and install the desktop launcher (Section 14).
4. Configure kiosk mode: desktop autologin, autostart, and verify it actually launches the app first (Section 15).
5. Optional polish, in any order: black-screen boot (Section 16), hide the taskbar (Section 17), passwordless power buttons (Section 18).

Result of Phase 1: a Pi that, from this point on, needs nothing manual to start working — every future power-on runs Phase 2 entirely on its own.

Phase 2 — Boot (every power-on)

1. Power is applied. Kernel boot text and the Raspberry Pi splash are suppressed (Section 16); the screen stays black.
2. `lxsession` starts the desktop session and launches `~/tech/dist/main` from its autostart entry (Section 15).
3. `main.py`'s startup sequence runs (Section 7): logging is wired up first, `config.json` is read, the four core objects (`ProductionModel`, `SerialReceiver`, `GPIOController`, `LogModel`) are constructed, `Backend` wires them together, the serial thread connects, and

`Main.qml` loads.

4. The Home screen appears in true fullscreen — this is the first thing anyone standing at the Pi actually sees.

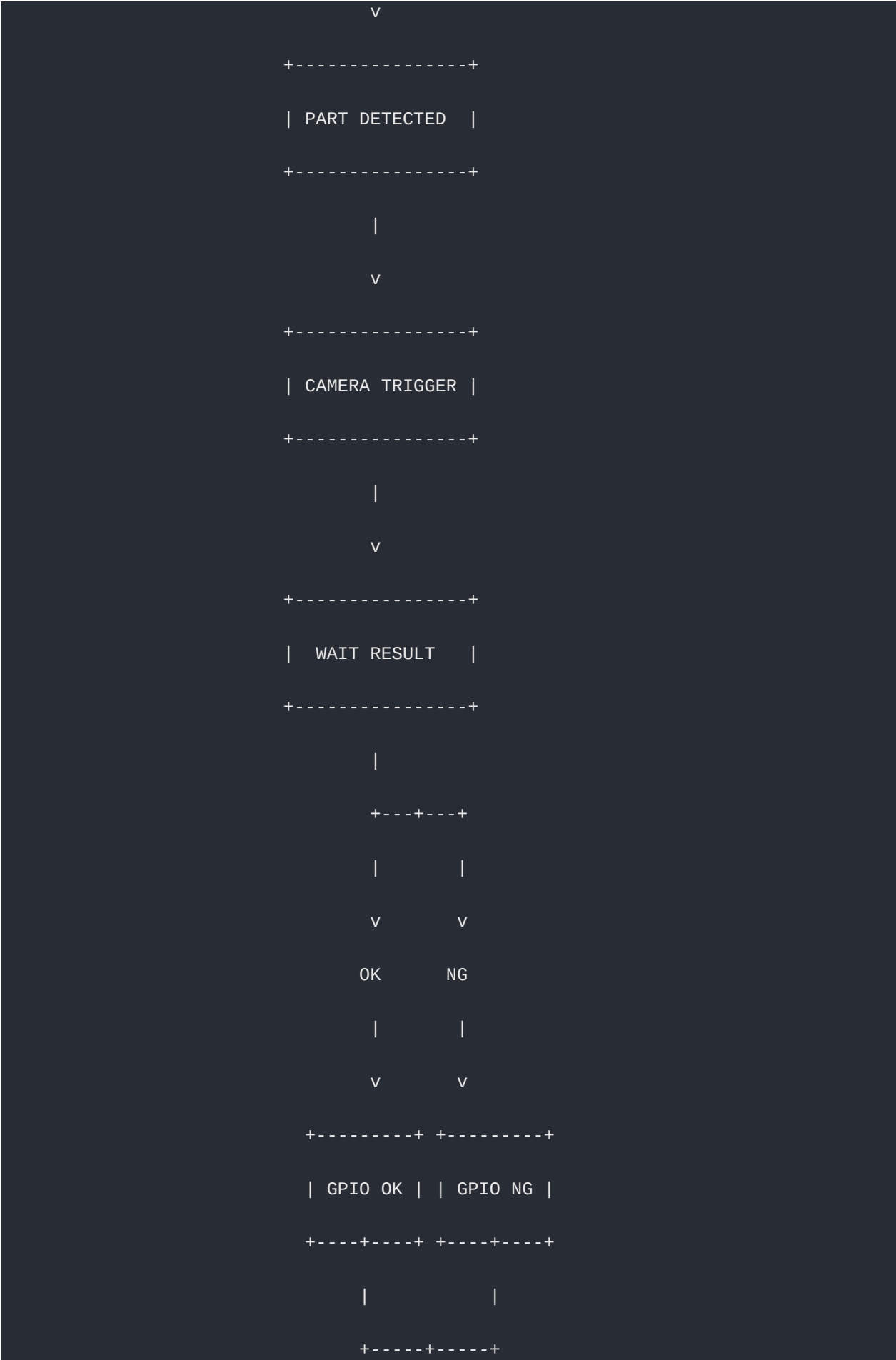
Phase 3 — Working Flow (the normal operating day)

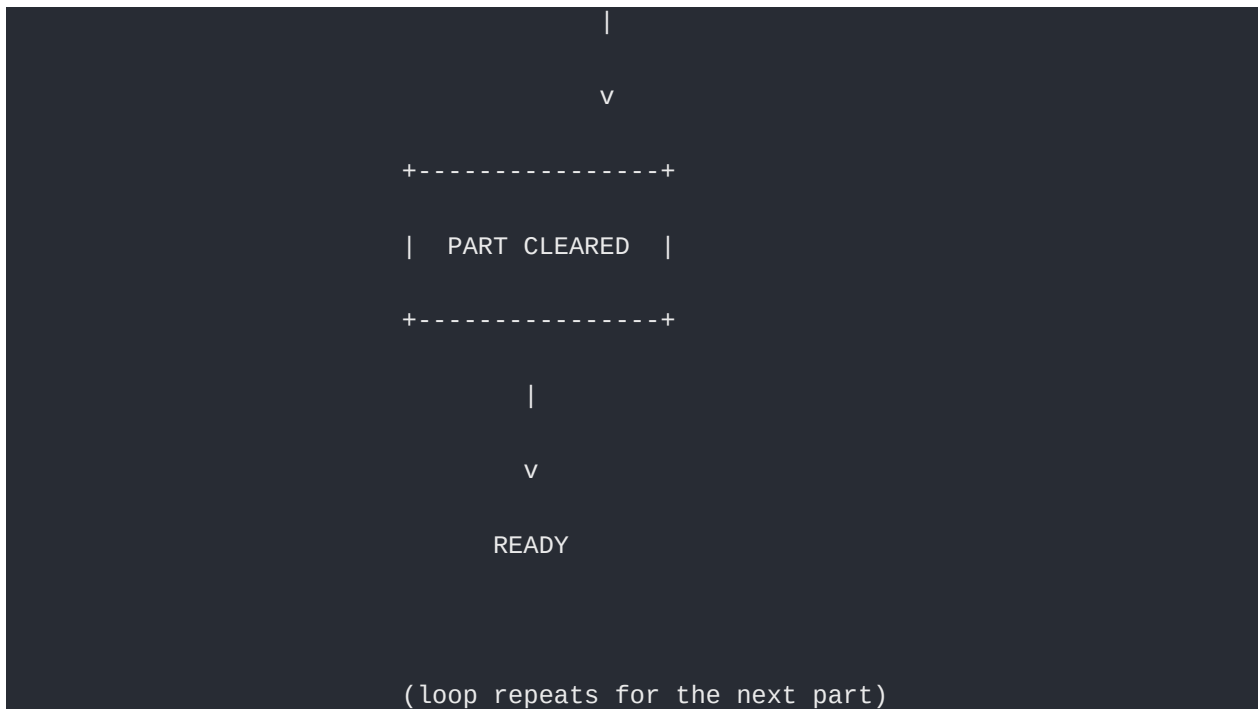
1. Operator taps a "Part N" button, then START — this calls `backend.selectScheme(N)` and `gpiocontroller.conveyor_on()` (Section 8's mapping table).
2. From here the inspection loop repeats on its own for every part that passes the sensor, with no operator action needed: sensor detects a part → camera is triggered → camera replies OK/NG → the result is counted, logged, and held on the OK/NG GPIO pins until the part clears the sensor (Section 8's full data-flow walkthrough; the exact pin logic is annotated in Section 6).
3. At any point the operator can: tap RESET to zero that part's on-screen tiles (Section 6's baseline mechanism), open Report for the four-part summary and full-history reset, or STOP/BACK to return Home and switch to a different part.
4. If something goes wrong during the day, `logs/app.log` (Section 19) has every status/error message printed since the app started.

Inspection Cycle State Machine

Step 2 above, expanded into the individual states each part moves through — this is the loop that repeats for every single part, with no operator involvement:







Mapped to the actual code: `INITIALIZING` is `GPIOController.__init__` starting the 50ms sensor poll; `READY/PART DETECTED` is `checkSensor()`'s edge detection; `CAMERA TRIGGER` is `Backend.handleSensor() → serialReceiver.trigger()`; `WAIT RESULT` through the `OK/NG` branch is `SerialReceiver._onLine()` parsing the camera's reply; `GPIO OK/GPIO NG` is `gpioController.set_ok()/set_ng()`; and `PART CLEARED` is the sensor-clears branch of `checkSensor()` that resets the pins. Every one of these is annotated line-by-line in Section 6's walkthroughs.

Phase 4 — Cleanup (every shutdown, reboot, or app exit)

This is the phase that does not appear as its own section elsewhere in this document — it is assembled here from the relevant pieces of Sections 6 and 9.

1. The operator taps the power button and confirms Shutdown or Reboot (Section 9), which calls `backend.shutdownSystem()` or `rebootSystem()` — running `sudo shutdown -h now` or `sudo reboot` under the passwordless rule from Section 18.
2. Before Qt actually tears the process down — whether from that `sudo` call terminating the app, or any other normal quit — `app.aboutToQuit` fires (wired in `main.py`, Section 7) and runs two cleanup calls in order:
 - `serialReceiver.disconnectSerial()` — stops the watchdog, signals the worker thread to stop, and waits up to 2s for it to quit cleanly, so the serial port is released rather than left open.

- `gpiocontroller.cleanup()` — stops the 50ms sensor poll timer, calls `all_off()` to drive every GPIO output (OK, NG, conveyor, conveyor-shorten) back to its inactive state, then releases the `gpiod` line request entirely.

3. Only after both of those complete does the underlying `sudo shutdown/reboot` command actually power down or restart the Pi.

Why this ordering matters: it is the reason the line controller never sees a GPIO output "stuck" in whatever state it happened to be in when the Pi loses power — every output is explicitly driven back off as part of shutdown, not left to whatever the hardware defaults to when 3.3V power disappears. `restartApp()` (app-only restart, no OS shutdown) would go through this exact same `aboutToQuit` cleanup before relaunching the binary, but it is not currently wired to a UI button (Section 6) — today, Shutdown and Reboot are the only two power actions reachable from the running app.

From here, the next power-on returns to Phase 2, and the cycle repeats indefinitely without anyone needing to touch Phase 1 again — that is the entire point of the kiosk setup this document describes.